

Inverse Problems in Computational Imaging

Contents

- [Content](#)
- [Introduction into inverse problems](#)
- [Image denoising](#)
- [General linear inverse problems](#)
- [Image deconvolution](#)
- [Inverse filter for deconvolution](#)
- [Wiener filter for deconvolution](#)
- [Richardson-Lucy deconvolution](#)
- [Half-quadratic splitting \(HQS\) method](#)
- [Alternating direction method of multipliers \(ADMM\)](#)
- [Unrolled optimization](#)
- [Deep SNR-estimation for Wiener filtering](#)

Computational Imaging

Content

- Introduction into inverse problems
- Image denoising
- General linear inverse problems
- Image deconvolution
- Inverse filter for deconvolution
- Wiener filter for deconvolution
- Richardson-Lucy deconvolution
- Half-quadratic splitting method
- Alternating direction method of multipliers
- Deep SNR-estimation for Wiener filtering
- Unrolled optimization

Introduction into inverse problems

An *inverse problem* represents the task of finding the input values to some function or process that produced certain output values which have been observed. Such problems are called *inverse*, as they start with the output and have the goal of finding the corresponding input.

Unfortunately, inverse problems often represent so-called *ill-posed problems*. A problem is considered to be ill-posed if one of the following properties holds:

- There is no solution to the problem,

- there is no unique solution, i.e., there are multiple possible inputs that explain the observed data,
- the solution does not change continuously w.r.t. to changes of the output.

A common approach to mitigate the difficulties involved with ill-posed inverse problems is to employ so-called *regularization methods* which usually use prior knowledge about the sought solution to guide the reconstruction process into the right direction.

Image denoising

The goal of image denoising is to recover an original image $s(\mathbf{x})$ after it has been affected by additive, signal-independent noise $n(\mathbf{x})$, which is a common general assumption for noise present in digitally acquired images:

$$g(\mathbf{x}) = s(\mathbf{x}) + n(\mathbf{x}) .$$

Note: The dependence on a spatial variable (e.g., like $\mathbf{x}$ for $g(\mathbf{x})$) is omitted for this chapter for the sake of clarity.

Note

The dependence on a spatial variable (e.g., like $\mathbf{x}$ for $g(\mathbf{x})$) is omitted for this chapter for the sake of clarity.

- The pixel-wise noise n_i follows a zero-mean i.i.d. normal distribution, i.e., $n_i \sim \mathcal{N}(0, \sigma^2)$ and
- the pixel s_i of the undistorted image follows an i.i.d. normal distribution with zero variance, i.e., $s_i \sim \mathcal{N}(s_i, 0)$.
- Hence, the pixel g_i of the observed image can be modelled via the sum of the two normal distributions, i.e., $g_i \sim \mathcal{N}(s_i, \sigma^2)$.

The per-pixel distribution of g is given by

$$p(g_i | s_i, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(g_i - s_i)^2}{2\sigma^2}} .$$

Due to the pixel-wise independence of the involved probabilities, the joint probability of the whole image is given by the product of the individual probabilities:

$$p(\mathbf{g} | \mathbf{s}, \sigma) = \prod_i p(g_i | s_i, \sigma) \propto e^{-\frac{\|\mathbf{g} - \mathbf{s}\|_2^2}{2\sigma^2}} .$$

Bayes' rule yields the posterior distribution for the sought undistorted image $\mathbf{s}$:

$$p(\mathbf{s} | \mathbf{g}, \sigma) = \frac{p(\mathbf{g} | \mathbf{s}, \sigma) p(\mathbf{s})}{p(\mathbf{g})} \propto p(\mathbf{g} | \mathbf{s}, \sigma) p(\mathbf{s}) ,$$

with the a priori probability $p(\mathbf{s})$ of the sought image $\mathbf{s}$.

Maximum a-posteriori (MAP) solution

The maximum a-posteriori (MAP) estimate of $\mathbf{s}$ can be obtained by maximizing (minimizing) the (negative) natural logarithm of the posterior distribution:

$$\begin{aligned} \hat{\mathbf{s}}_{\text{MAP}} &= \arg \min_{\mathbf{s}} -\log(p(\mathbf{s} | \mathbf{g}, \sigma)) \\ &= \arg \min_{\mathbf{s}} -\log p(\mathbf{g} | \mathbf{s}, \sigma) - \log p(\mathbf{s}) \\ &= \arg \min_{\mathbf{s}} \frac{1}{2\sigma^2} \|\mathbf{g} - \mathbf{s}\|_2^2 - \log p(\mathbf{s}) \\ &= \arg \min_{\mathbf{s}} \frac{1}{2\sigma^2} \|\mathbf{g} - \mathbf{s}\|_2^2 + \Gamma(\mathbf{s}) , \end{aligned}$$

with $\Gamma(\mathbf{s})$ denoting the negative natural logarithm of the prior on $\mathbf{s}$.

If no information about the prior $p(\mathbf{s})$ is exploited, every pixel value has to be considered equally likely, i.e., $p(s_i) = 1$, resulting in a trivial but useless solution for the estimate of $\mathbf{s}$:

$$\hat{\mathbf{s}}_{\text{flat prior}} = \arg \min_{\mathbf{s}} \frac{1}{2\sigma^2} \|\mathbf{g} - \mathbf{s}\|_2^2 = \mathbf{g}.$$

We will get to know several suitable priors later on in this chapter but we want to stress one important point here:

Any algorithm that successfully denoises an image from normally distributed noise can be interpreted (and used) as a solution to the problem just described, i.e.,

$$\arg \min_{\mathbf{s}} \frac{1}{2\sigma^2} \|\mathbf{g} - \mathbf{s}\|_2^2 + \Gamma(\mathbf{s}).$$

General linear inverse problems

For the general linear inverse problem, the vectorized image formation process is given by:

$$\mathbf{g} = \mathbf{H}\mathbf{s} + \mathbf{n},$$

where $\mathbf{g} \in \mathbb{R}^M$ denotes the M observations or measurements which are the result of the matrix-vector multiplication of the sought latent image $\mathbf{s} \in \mathbb{R}^N$ with the so-called *measurement matrix* $\mathbf{H} \in \mathbb{R}^{M \times N}$ and the term $\mathbf{n} \in \mathbb{R}^M$ represents additive, signal-independent noise.

Bayesian perspective of inverse problems

Interpret as random vectors:

$$\tilde{\mathbf{s}} \sim \mathcal{N}(\mathbf{s}, 0),$$

$$\tilde{\mathbf{n}} \sim \mathcal{N}(0, \sigma^2),$$

$$\tilde{\mathbf{g}} \sim \mathcal{N}(\mathbf{H}\mathbf{s}, \sigma^2)$$

For probability of observation $\mathbf{g}$ it holds:

$$p(\mathbf{g}|\mathbf{s}, \sigma) \propto \exp\left(-\frac{\|\mathbf{g} - \mathbf{H}\mathbf{s}\|_2^2}{2\sigma^2}\right)$$

According to Bayes' rule:

$$\underbrace{p(\mathbf{s}|\mathbf{g}, \sigma)}_{\text{posterior}} = \frac{p(\mathbf{g}|\mathbf{s}, \sigma) \cdot p(\mathbf{s})}{p(\mathbf{g})} \propto \underbrace{p(\mathbf{g}|\mathbf{s}, \sigma)}_{\text{image formation model}} \cdot \underbrace{p(\mathbf{s})}_{\text{prior}}.$$

Maximum-a-posteriori (MAP) solution:

$$\begin{aligned} \hat{x}_{\text{MAP}} &= \arg \max_{\mathbf{s}} p(\mathbf{s}|\mathbf{g}, \sigma) \\ &= \arg \max_{\mathbf{s}} \log p(\mathbf{s}|\mathbf{g}, \sigma) \\ &= \arg \min_{\mathbf{s}} -\log p(\mathbf{s}|\mathbf{g}, \sigma) \\ &= \arg \min_{\mathbf{s}} -\log p(\mathbf{g}|\mathbf{s}, \sigma) - \log p(\mathbf{s}) \\ &= \arg \min_{\mathbf{s}} \underbrace{\frac{\|\mathbf{g} - \mathbf{H}\mathbf{s}\|_2^2}{2\sigma^2}}_{\text{data fidelity term}} + \underbrace{\Psi(\mathbf{s})}_{\text{regularizer}}. \end{aligned}$$

The choice of image priors / regularizers depends on the imaging task, i.e., the nature of the images that are to be recovered. Examples are:

- Blurry images $\rightarrow$ promote smoothness $\Psi(\mathbf{s}) = \left\| \underbrace{\Delta}_{\text{Laplace operator}} \mathbf{s} \right\|_2$
- Sparse images (e.g., stars) $\rightarrow$ promote sparsity $\Psi(\mathbf{s}) = \|\mathbf{s}\|_1$

- Natural images $\rightarrow$ promote sparse gradients $\text{TV}(\mathbf{s})$

Total Variation

The intuition behind total variation is that in natural images, regions of almost constant intensities are separated by sharp edges. Hence, the gradient of such images can be assumed to be sparse.

The gradient is calculated by means of convolutions with the finite difference operators in x - and y -direction:

- Finite difference in x -direction: $d_x * s = \begin{pmatrix} 0 & 0 & 0 \\ 0 & -1 & 1 \\ 0 & 0 & 0 \end{pmatrix} * s = \mathbf{D}_x \mathbf{s}$
- Finite difference in y -direction: $d_y * s = \begin{pmatrix} 0 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 1 & 0 \end{pmatrix} * s = \mathbf{D}_y \mathbf{s}$

- Anisotropic:

$$\begin{aligned} \text{TV}(\mathbf{s}) &= \|\mathbf{D}_x \mathbf{s}\|_1 + \|\mathbf{D}_y \mathbf{s}\|_1 \\ &= \sum_{i=1}^N |(\mathbf{D}_x \mathbf{s})_i| + |(\mathbf{D}_y \mathbf{s})_i| = \sum_{i=1}^N \sqrt{(\mathbf{D}_x \mathbf{s})_i^2} + \sum_{i=1}^N \sqrt{(\mathbf{D}_y \mathbf{s})_i^2} \end{aligned}$$

- Isotropic:

$$\text{TV}(\mathbf{s}) = \sum_{i=1}^N \left\| \begin{bmatrix} (\mathbf{D}_x \mathbf{s})_i \\ (\mathbf{D}_y \mathbf{s})_i \end{bmatrix} \right\|_2 = \sum_{i=1}^N \sqrt{(\mathbf{D}_x \mathbf{s})_i^2 + (\mathbf{D}_y \mathbf{s})_i^2},$$

with $\begin{bmatrix} \mathbf{a} \\ \mathbf{b} \end{bmatrix}$ denoting a concatenation of the vectors $\mathbf{a}$ and $\mathbf{b}$ and $(\mathbf{D}_x \mathbf{s})_i$ denoting the i -th element of the vector resulting from $\mathbf{D}_x \mathbf{s}$.

Image deconvolution

Image deconvolution deals with the problem of reconstructing an original image s after it has been affected by a linear shift-invariant image degradation and additive noise, which together forms the observed image g .

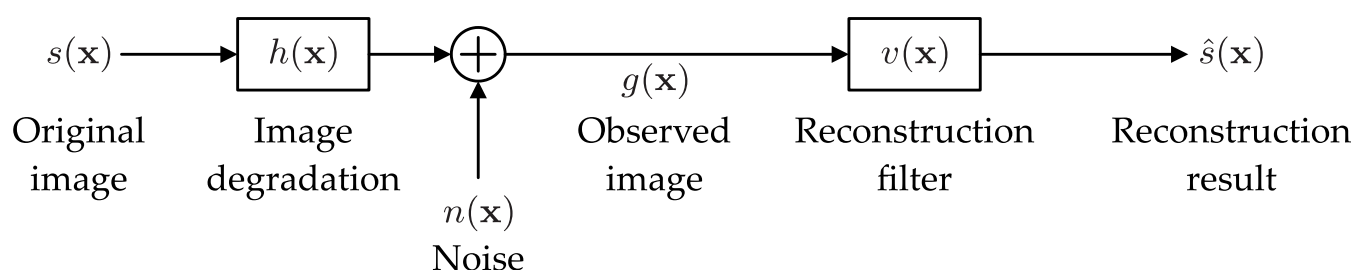
With the impulse response h of the image degradation, the observation can be modeled as

$$g = s * h + n,$$

with the additive noise n .

The goal is to define a reconstruction filter v that, when applied to the observation g , yields a reconstruction $\hat{s}$ matching the latent input s as closely as possible.

The following figure visualizes the signal model of image deconvolution:



Methods and approaches dealing with deconvolution can be divided into two groups:

1. *Blind* deconvolution: The kernel h is assumed to be unknown and also has to be recovered / estimated.
2. *Non-blind* deconvolution: The kernel h is assumed to be known.

Note that there is some research which focuses on estimating the kernel h and then employs non-blind deconvolution methods to perform the deconvolution.

According to the convolution theorem of the Fourier transform, equation (???) can be expressed as: $g = \mathcal{F}^{-1}\{\mathcal{F}\{h\} \cdot \mathcal{F}\{s\}\} + n$

Important note: Equations (???) and (???) only result in the same numbers, if the convolution is performed with circlar boundary conditions, i.e., when the convolution kernel wraps around the image borders.

Duality between signal processing perspective and algebraic perspective

Since the convolution is a linear operator, it can also be expressed in terms of a matrix vector multiplication: $g = h * s \Leftrightarrow \mathbf{g} = \mathbf{H}\mathbf{s}$, with a square circulant Toeplitz matrix $\mathbf{H} \in \mathbb{R}^{N \times N}$ implementing the convolution and $\mathbf{g}, \mathbf{s} \in \mathbb{R}^N$ representing the vectorized forms of the corresponding images g and s .

Inverse filter for deconvolution

The inverse filter represents the straightforward approach of solving equation (???) for s while neglecting the noise n : $\hat{s}_{\text{inv}} = \mathcal{F}^{-1}\left\{\frac{\mathcal{F}\{g\}}{\mathcal{F}\{h\}}\right\}$.

Properties of the inverse filter:

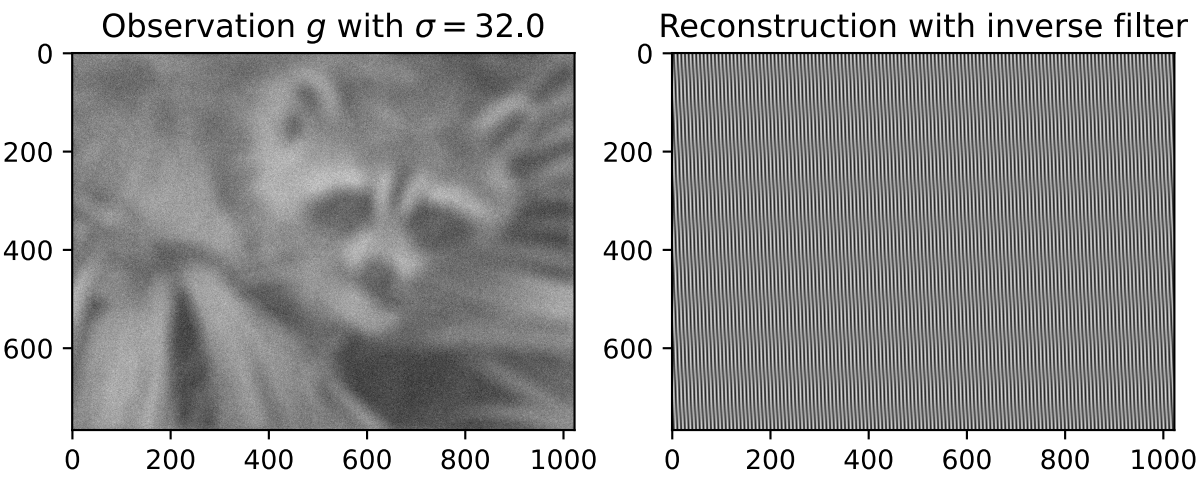
- computationally efficient,
- problematic for small values of $\mathcal{F}\{h\}$, (amplifies noise).

Unfortunately, the second point is true for most practically relevant point spread functions.

Example inverse filter reconstruction results for increasing noise levels

```
interact(lambda i: showFig('figures/6/inv_filter_res_',i,'.svg',800,50),
i=widgets.IntSlider(min=(min_i:=1),max=(max_i:=6), step=1, value=(max_i if book else min_i)))
```

i  6



```
<function __main__.<lambda>(i)>
```

Wiener filter for deconvolution

The Wiener filter has been proposed by Norbert Wiener in 1949 and minimizes the mean squared error between the original image s and the reconstruction $\hat{s}$.

The filter is obtained by assuming the involved images to be weakly stationary stochastic processes so that they can be described via their power spectral densities (not covered here).

In the Fourier-domain, the Wiener filter is given by

$$V(\mathbf{f}) = \frac{H(\mathbf{f})^*}{|H(\mathbf{f})|^2 + \underbrace{\frac{S_{nn}(\mathbf{f})}{S_{ss}(\mathbf{f})}}_{=:SNR^{-1}(\mathbf{f})}},$$

with $H(\mathbf{f})^*$ denoting the complex conjugate of $H(\mathbf{f})$ and $S_{nn}(\mathbf{f})$, $S_{ss}(\mathbf{f})$ denoting the power spectral densities of the noise n and the original image s . The quotient $\frac{S_{ss}(\mathbf{f})}{S_{nn}(\mathbf{f})}$ represents the signal-to-noise ration $SNR(\mathbf{f})$ between s and n .

For frequencies with high signal-to-noise ratios, the Wiener filter resembles the inverse filter and for low signal-to-noise ratios, it pulls yields values close to zero in order not to amplify any noise.

One of the main practical problems of the Wiener filter is the dependence on the signal-to-noise ratio $SNR(\mathbf{f})$ which can not be determined exactly as it involves the power spectral density of the unknown, original image s .

A common heuristic for providing the signal-to-noise ratio is

$$SNR_{\text{heur}}(\mathbf{f}) = \frac{1}{\|\mathbf{f}\|_2},$$

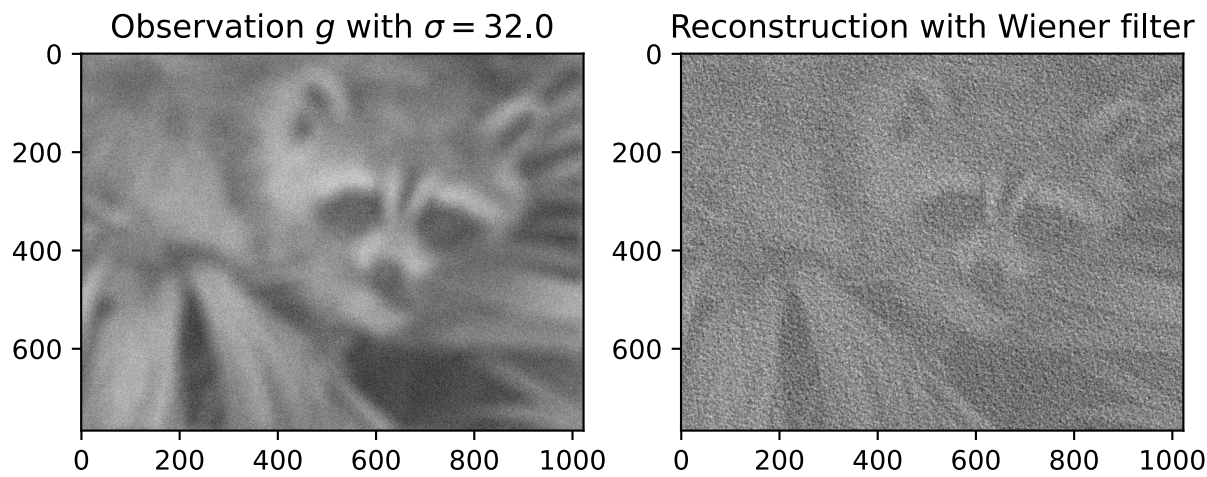
i.e., to steadily decrease the filtering for higher spatial frequencies as noise is typically dominated by high spatial frequencies.

Later we will see another method for estimating the signal-to-noise ratio that takes the actual processed image g into account.

Example Wiener filter reconstruction results for increasing noise levels with SNR-heuristics

```
interact(lambda i: showFig('figures/6/wiener_filter_res_',i,'.svg',800,50),
i=widgets.IntSlider(min=(min_i:=1),max=(max_i:=6), step=1, value=(max_i if book else min_i)))
```

i  6



```
<function __main__.<lambda>(i)>
```

Richardson-Lucy deconvolution

The Richardson-Lucy deconvolution algorithm (RL-algorithm) has been simultaneously and independently proposed by William Richardson and Leon Lucy in the 1970s.

It is based on the assumption that the noise affecting digitally acquired images is dominated by the so-called *shot noise* caused by the stochastic nature of the number of arriving photons.

We again consider the image formation process

$$g = s * h + n,$$

with the observed image g , the sought original image s , the convolution kernel h and the additive noise n .

The probability $p(g)$ that a single pixel collects g photons during one exposure period can be modelled by means of a Poisson distribution:

$$p(g) = e^{-\lambda} \cdot \frac{\lambda^g}{g!},$$

with λ denoting the expected number of photons collected during one exposure.

In order to obtain a noise-free image, one would need to obtain λ by means of the measured g photons (precisely, photoelectrons).

Since the joint probability of two independent events is the product of their individual probabilities, the joint probability of recording the intensities g of a whole image can be expressed via:

$$P(g) = \prod_{(x,y) \in \Omega_g} e^{-\Lambda_{x,y}} \cdot \frac{\Lambda_{x,y}^{g_{x,y}}}{g_{x,y}!},$$

with Ω_g denoting the spatial support of the image g , $\Lambda_{x,y}$ denoting the value of the sought noise-free image Λ at position (x, y) and $g_{x,y}$ denoting the pixel value of the image g at position (x, y) . All operations are considered element-wise.

To improve readability, we will omit the pixel index (x, y) from now on.

To obtain the noise-free image, we have to guess the hidden expected values $\hat{\Lambda}$ that match the observed image g as closely as possible.

We can achieve this by finding a $\hat{\Lambda}$ that maximizes $P(g)$, or, equivalently, that maximizes $\log P(g)$:

$$\begin{aligned}\log P(g) &= \log \left(\prod e^{-\Lambda} \cdot \frac{\Lambda^g}{g!} \right) \\ &= \sum \log \left(e^{-\Lambda} \cdot \frac{\Lambda^g}{g!} \right) \\ &= \sum \left(\log e^{-\Lambda} + g \log \Lambda - \log g! \right) \\ &= - \sum (\Lambda - g \log \Lambda + \log g!),\end{aligned}$$

with $\log$ denoting the natural logarithm.

The last term in the summation, i.e., $\log g!$, can be ignored since it does not depend on Λ .

The noise-free image Λ is the result of

$$\Lambda = s * h,$$

i.e., the convolution of the noise-free, original image s with the convolution kernel h .

And the observed image g is the result Of

$$g = s * h + n,$$

i.e., Λ with additive noise n .

These two expressions can be inserted into the equation of $\log P(g)$ in order to get to our optimization goal of finding an $\hat{s}$ that maximizes $\log P(g)$, i.e., minimizes $-\log P(g)$:

$$\hat{s} = \arg \min_s \underbrace{\sum (s * h - g \log(s * h))}_{=: J(s)},$$

with the optimization functional $J(s)$.

To perform the optimization, we can employ gradient descent on $J(s)$. The gradient $\nabla J = \frac{\partial J(s)}{\partial s}$ of $J(s)$ w.r.t. s is given by:

$$\nabla J(s) = \left(1 - \frac{g}{s * h} \right) * h^\top,$$

with element-wise multiplication and division.

For an initial guess of $\hat{s}_0 := g$, the gradient descent iteration for the Richardson-Lucy algorithm is

$$\hat{s}_{i+1} = \hat{s}_i - \eta \nabla J(\hat{s}_i) = \hat{s}_i - \eta \left(1 - \frac{g}{\hat{s}_i * h} \right) * h^\top.$$

Multiplicative version of the Richardson-Lucy algorithm

The most implemented version of the RL-algorithm is the so-called *multiplicative RL-algorithm*.

It relies on the convolution kernel h being normalized, i.e., $\sum_{(i,j)} h_{i,j} = 1$.

This also implies that $1 * h^\top = 1$.

The multiplicative RL-algorithm is derived by equating $J(s)$ to zero. This results in:

$$1 = \frac{g}{s * h} * h^\top.$$

Since $J(\mathbf{s})$ is a convex function, it decreases with every step of gradient descent until it does not change anymore, i.e., until it holds

$$\frac{\hat{s}_{i+1}}{\hat{s}_i} = 1.$$

We use this expression to substitute 1 in the equation before and arrive at the sought multiplicative formulation of the RL-algorithm:

$$\begin{aligned} \frac{\hat{s}_{i+1}}{\hat{s}_i} &= \frac{g}{\hat{s} * h} * h^\top \Leftrightarrow \\ \hat{s}_{i+1} &= \hat{s}_i \frac{g}{\hat{s}_i * h} * h^\top. \end{aligned}$$

Comments on the Richardson-Lucy algorithm

The RL-algorithm is implemented in many open source or commercial image processing libraries.

The main problem of the RL-algorithm is that it starts to amplify noise after its (usually quick) convergence. Hence, the intermediate results have to be continuously inspected or some sort of regularization has to be incorporated into the optimization functional J .

Half-quadratic splitting (HQS) method

This section introduces a flexible, powerful and intuitive iterative approach, the half-quadratic splitting (HQS) method, for solving regularized inverse problems formulated like [\eqref{eq:map_solution}](#).

We start with some general considerations and then apply HQS to the inverse problem of deconvolution.

We assume the following image formation model:

$$\mathbf{g} = \mathbf{H}\mathbf{s} + \mathbf{n},$$

with $\mathbf{s} \in \mathbb{R}^N$ denoting the unknown vector, $\mathbf{g} \in \mathbb{R}^M$ representing the observations, the additive noise $\mathbf{n} \in \mathbb{R}^M$ and the matrix $\mathbf{H} \in \mathbb{R}^{M \times N}$ encoding the linear image formation model.

Equation [\eqref{eq:map_solution}](#) leads us to the general formulation of regularized inverse problems in the field of computational imaging:

$$\hat{\mathbf{s}} = \arg \min_{\mathbf{s}} \underbrace{\frac{1}{2} \|\mathbf{H}\mathbf{s} - \mathbf{g}\|_2^2}_{\text{data fidelity term}} + \underbrace{\lambda \Psi(\mathbf{s})}_{\text{regularizer}}.$$

The data fidelity term ensures that the sought solution $\hat{\mathbf{s}}$ matches the observed data $\mathbf{g}$ when fed through the image formation process (modelled by $\mathbf{H}$). The regularization operator $\Psi : \mathbb{R}^N \rightarrow \mathbb{R}$ models prior knowledge about the unknown original data $\mathbf{s}$. The scalar parameter λ balances between the data fidelity term and the regularization term and hence $\lambda \in [0, 1]$.

Trying to directly solve [\eqref{eq:general_inverse_problem}](#), e.g., via gradient descent, often does not work well. Reasons are poor convergence or difficulties in finding an efficient way to calculate the gradient. Even worse, whenever we change the regularizer, we will have to re-write the optimization program again.

Hence, we rewrite [\eqref{eq:general_inverse_problem}](#) to:

$$\begin{aligned} \arg \min_{\mathbf{s}} \quad & \underbrace{\frac{1}{2} \|\mathbf{H}\mathbf{s} - \mathbf{g}\|_2^2}_{=:f(\mathbf{s})} + \underbrace{\lambda \Psi(\mathbf{z})}_{=:r(\mathbf{z})} \\ \text{subject to} \quad & \mathbf{s} - \mathbf{z} = \mathbf{0}. \end{aligned}$$

By directly passing the derivative $\mathbf{D}\mathbf{s}$ to the regularizer, we get:

$$\arg \min_{\mathbf{s}} \underbrace{\frac{1}{2} \|\mathbf{H}\mathbf{s} - \mathbf{g}\|_2^2}_{=:f(\mathbf{s})} + \underbrace{\lambda \Psi(\mathbf{z})}_{=:r(\mathbf{z})}$$

subject to $\mathbf{D}\mathbf{s} - \mathbf{z} = \mathbf{0}$.

We introduced a so-called *slack variable* $\mathbf{z} \in \mathbb{R}^O$ which allows us to separate the data fidelity term and the regularization term so that they do not depend on the same variable anymore.

Obviously, $\mathbf{s}$ and $\mathbf{z}$ are still linked by the constraint $\mathbf{D}\mathbf{s} - \mathbf{z} = \mathbf{0}$.

For now, we assume $\mathbf{D} \in \mathbb{R}^{O \times N}$ to represent the identity matrix (i.e., it does not introduce any changes and can be ignored for now) - it will come back into play later on.

We now include the constraint of \eqref{eq:hqs_1} directly in the main optimization objective term via a penalty term:

$$L_\rho(\mathbf{s}, \mathbf{z}) = f(\mathbf{s}) + r(\mathbf{z}) + \frac{\rho}{2} \|\mathbf{D}\mathbf{s} - \mathbf{z}\|_2^2, \quad \text{with } \rho > 0.$$

Intuitively, setting ρ to a large value leads to the same results for minimizing \eqref{eq:hqs_1} and \eqref{eq:hqs_2}. The benefit of this reformulation is that we can perform gradient descent for $\mathbf{s}$ and $\mathbf{z}$ in an alternating fashion.

$$\begin{aligned} \mathbf{s} &\leftarrow \text{prox}_{f,\rho}(\mathbf{z}) = \arg \min_{\mathbf{s}} L_\rho(\mathbf{s}, \mathbf{z}) = \arg \min_{\mathbf{s}} f(\mathbf{s}) + \frac{\rho}{2} \|\mathbf{D}\mathbf{s} - \mathbf{z}\|_2^2, \\ \mathbf{z} &\leftarrow \text{prox}_{r,\rho}(\mathbf{D}\mathbf{s}) = \arg \min_{\mathbf{z}} L_\rho(\mathbf{s}, \mathbf{z}) = \arg \min_{\mathbf{z}} r(\mathbf{z}) + \frac{\rho}{2} \|\mathbf{D}\mathbf{s} - \mathbf{z}\|_2^2. \end{aligned}$$

Again, the important benefit of this formulation is that we can update $\mathbf{s}$ and $\mathbf{z}$ separately. We will see that this approach allows us to easily experiment with different regularizers (this is sometimes also referred to as a *plug-and-play* formulation).

Excursion on proximal operators

So-called *proximal operators* are used in *proximal algorithms* to solve

- large-scale,
- non-smooth and
- constrained convex optimization problems.

Similar to calculating the gradient of the target function f in gradient descent, one evaluates the corresponding proximal operator to take one step into the direction of the sought minimum.

Definition of a proximal operator

Assume $f : \mathbb{R}^n \rightarrow \mathbb{R} \cup \{+\infty\}$ to be a closed proper convex function, i.e., with its *epigraph*

$$\text{epi}_t f := \{(\mathbf{x}, t) \in \mathbb{R}^n \times \mathbb{R} \mid f(\mathbf{x}) \leq t\}$$

being a nonempty closed convex set.

The proximal operator $\text{prox}_f(\mathbf{v}) : \mathbb{R}^n \rightarrow \mathbb{R}^n$ of f is defined as

$$\text{prox}_f(\mathbf{v}) = \arg \min_{\mathbf{x}} \left(f(\mathbf{x}) + \frac{1}{2} \|\mathbf{x} - \mathbf{v}\|_2^2 \right).$$

The minimized term on the right-hand side is strongly convex and not infinite everywhere, so it has a unique solution for every $\mathbf{v} \in \mathbb{R}^n$.

Often the proximal operator for the scaled function λf , with $\lambda > 0$, is of interested, which we denote as:

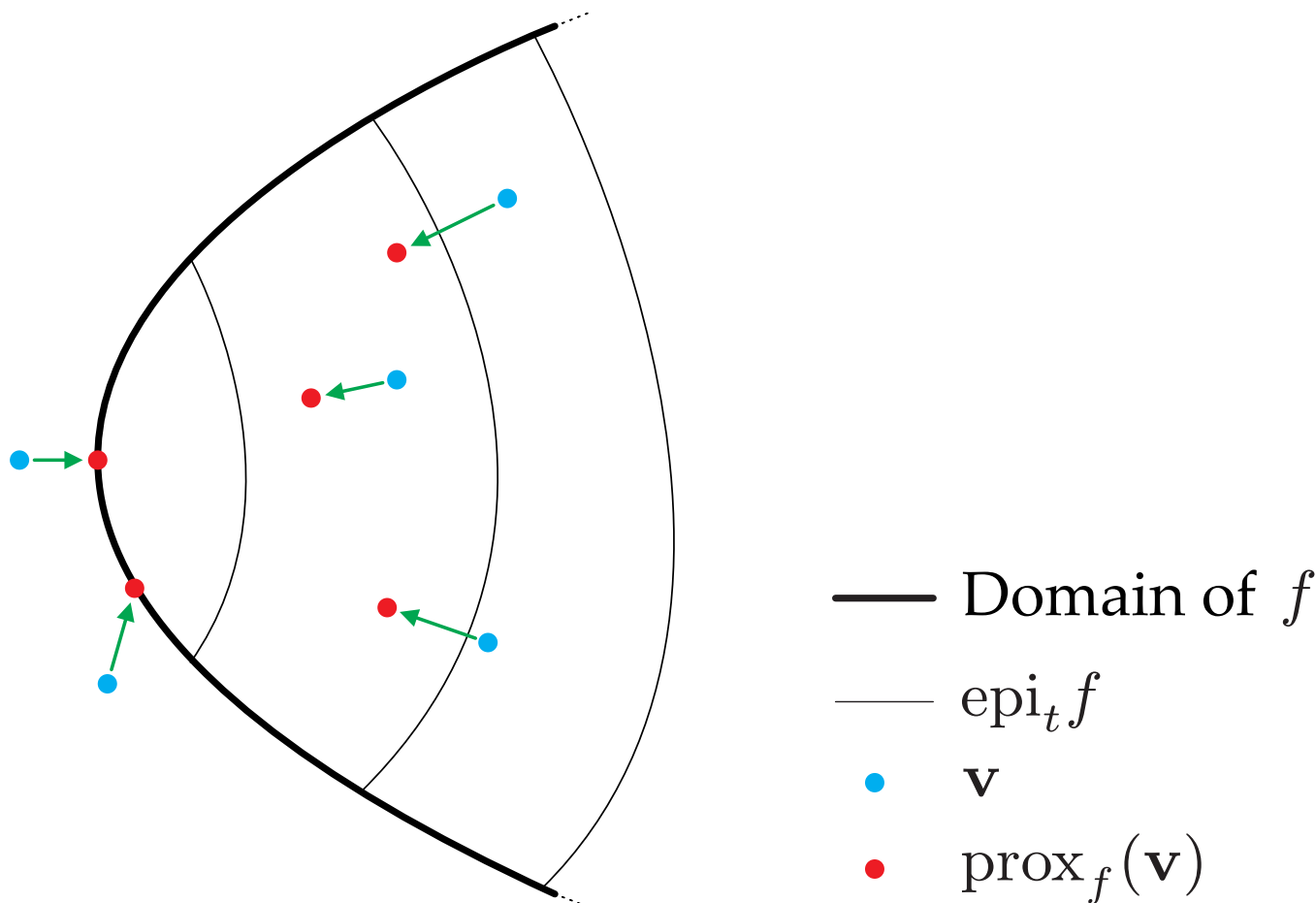
$$\text{prox}_{f,\lambda}(\mathbf{v}) = \arg \min_{\mathbf{x}} \left(f(\mathbf{x}) + \frac{1}{2} \lambda \|\mathbf{x} - \mathbf{v}\|_2^2 \right).$$

Interpretation of proximal operators

Intuitively, a proximal operator $\text{prox}_f(\mathbf{v})$ yields a point that is a compromise between minimizing f and being near to $\mathbf{v}$.

In the scaled version $\text{prox}_{f,\lambda}(\mathbf{v})$, λ acts like a relative weight between the two terms.

The effect of a proximal operator can be visualized as follows:



Points $\mathbf{v}$ inside the domain of f are moved by the proximal operator so that they stay in the domain and get closer to the minimum of f .

Points $\mathbf{v}$ outside the domain of f are moved onto the boundary of f and in the direction of the minimum of f .

Proximal operators can also be interpreted as a kind of gradient descent step in the sense that

$$\text{prox}_{f,\lambda}(\mathbf{v}) \approx \mathbf{v} - \lambda \nabla f(\mathbf{v}),$$

when λ is small and f is differentiable.

HQS for deconvolution

We now again consider the inverse problem of deconvolution with circular boundary conditions.

Here, the matrix $\mathbf{H}$ is the square circulant Toeplitz matrix $\mathbf{H} \in \mathbb{R}^{N \times N}$ representing a 2D-convolution of the input image s with the convolution kernel h .

Revise the duality between the signal processing formulation and the algebraic formulation:

$$\begin{aligned} h * s &= \mathcal{F}^{-1} \{ \mathcal{F} \{ h \} \cdot \mathcal{F} \{ s \} \} \Leftrightarrow \mathbf{H} \mathbf{s}, \\ \mathcal{F}^{-1} \{ \mathcal{F} \{ h \}^* \cdot \mathcal{F} \{ s \} \} &\Leftrightarrow \mathbf{H}^T \mathbf{s}, \\ \mathcal{F}^{-1} \left\{ \frac{\mathcal{F} \{ g \}}{\mathcal{F} \{ h \}} \right\} &\Leftrightarrow \mathbf{H}^{-1} \mathbf{g}. \end{aligned}$$

HQS with total variation and denoising regularizers

The initial formulation [\eqref{eq:hqs_1}](#) of HQS depends on which regularizer we employ.

For total variation this is:

$$\begin{aligned} \arg \min_{\mathbf{s}} \quad & \underbrace{\frac{1}{2} \|\mathbf{H} \mathbf{s} - \mathbf{g}\|_2^2}_{=: f(\mathbf{s})} + \underbrace{\lambda \|\mathbf{z}\|_1}_{=: r(\mathbf{z})} \\ \text{subject to} \quad & \mathbf{D} \mathbf{s} - \mathbf{z} = \mathbf{0}, \end{aligned}$$

with $\mathbf{D} = [\mathbf{D}_x^\top \mathbf{D}_y^\top]^\top \in \mathbb{R}^{2N \times N}$ representing the finite difference operator for calculating the gradients of $\mathbf{s}$ in x - and y -direction.

Note

The vector $\mathbf{z} \in \mathbb{R}^{2N}$ has to be twice as large as $\mathbf{s} \in \mathbb{R}^N$ in order to store the two gradient values in x - and y -direction for every input pixel.

In the more general case, we use a regularizer Ψ projecting an image onto the set of feasible natural images:

$$\arg \min_{\mathbf{s}} \underbrace{\frac{1}{2} \|\mathbf{H}\mathbf{s} - \mathbf{g}\|_2^2}_{=:f(\mathbf{s})} + \underbrace{\lambda \Psi(\mathbf{z})}_{=:r(\mathbf{z})}$$

subject to $\mathbf{s} - \mathbf{z} = \mathbf{0}$.

Here the matrix $\mathbf{D}$ represents the identity matrix which is why it can be omitted.

Efficient implementation of s -update

For obtaining the s -update, we have to derive the proximal operator $\text{prox}_{f,\rho}$:

$$\text{prox}_{f,\rho}(\mathbf{z}) = \arg \min_{\mathbf{s}} f(\mathbf{s}) + \frac{\rho}{2} \|\mathbf{D}\mathbf{s} - \mathbf{z}\|_2^2 = \arg \min_{\mathbf{s}} \frac{1}{2} \|\mathbf{H}\mathbf{s} - \mathbf{g}\|_2^2 + \frac{\rho}{2} \|\mathbf{D}\mathbf{s} - \mathbf{z}\|_2^2.$$

Hence, we have to derive the gradient of that equation with respect to $\mathbf{s}$.

We iteratively expand that equation as follows:

$$\begin{aligned} & \frac{1}{2} \|\mathbf{H}\mathbf{s} - \mathbf{g}\|_2^2 + \frac{\rho}{2} \|\mathbf{D}\mathbf{s} - \mathbf{z}\|_2^2 \\ &= \frac{1}{2} (\mathbf{H}\mathbf{s} - \mathbf{g})^\top (\mathbf{H}\mathbf{s} - \mathbf{g}) + \frac{\rho}{2} (\mathbf{D}\mathbf{s} - \mathbf{z})^\top (\mathbf{D}\mathbf{s} - \mathbf{z}) \\ &= \frac{1}{2} (\mathbf{s}^\top \mathbf{H}^\top \mathbf{H} \mathbf{s} - 2\mathbf{s}^\top \mathbf{H}^\top \mathbf{g} + \mathbf{g}^\top \mathbf{g}) + \frac{\rho}{2} (\mathbf{s}^\top \mathbf{D}^\top \mathbf{D} \mathbf{s} - 2\mathbf{s}^\top \mathbf{D}^\top \mathbf{z} + \mathbf{z}^\top \mathbf{z}). \end{aligned}$$

The sought gradient is

$$\mathbf{H}^\top \mathbf{H} \mathbf{s} - \mathbf{H}^\top \mathbf{g} + \rho \mathbf{D}^\top \mathbf{D} \mathbf{s} - \rho \mathbf{D}^\top \mathbf{z}.$$

This expression can now be equated to zero and solved for $\mathbf{s}$.

The single steps are:

$$\begin{aligned} \mathbf{H}^\top \mathbf{H} \mathbf{s} - \mathbf{H}^\top \mathbf{g} + \rho \mathbf{D}^\top \mathbf{D} \mathbf{s} - \rho \mathbf{D}^\top \mathbf{z} &\stackrel{!}{=} \mathbf{0} \\ \mathbf{H}^\top \mathbf{H} \mathbf{s} + \rho \mathbf{D}^\top \mathbf{D} \mathbf{s} &\stackrel{!}{=} \mathbf{H}^\top \mathbf{g} + \rho \mathbf{D}^\top \mathbf{z} \\ (\mathbf{H}^\top \mathbf{H} + \rho \mathbf{D}^\top \mathbf{D}) \mathbf{s} &\stackrel{!}{=} \mathbf{H}^\top \mathbf{g} + \rho \mathbf{D}^\top \mathbf{z} \\ \mathbf{s} &\stackrel{!}{=} (\mathbf{H}^\top \mathbf{H} + \rho \mathbf{D}^\top \mathbf{D})^{-1} (\mathbf{H}^\top \mathbf{g} + \rho \mathbf{D}^\top \mathbf{z}). \end{aligned}$$

This yields a first formulation for the sought proximal operator:

$$\text{prox}_{f,\rho}(\mathbf{z}) = (\mathbf{H}^\top \mathbf{H} + \rho \mathbf{D}^\top \mathbf{D})^{-1} (\mathbf{H}^\top \mathbf{g} + \rho \mathbf{D}^\top \mathbf{z}).$$

Special case of total variation regularizer

For the TV regularizer, the matrix $\mathbf{D}$ represents the finite difference operator. The matrix-vector multiplications involved in the proximal operator, i.e., $\mathbf{H}\mathbf{s}$ and $\mathbf{D}\mathbf{s} = [\mathbf{D}_x^\top \mathbf{D}_y^\top]^\top \mathbf{s}$ (and also their adjoint correspondencies $\mathbf{H}^\top \mathbf{g}$ and $\mathbf{D}^\top \mathbf{z} = [\mathbf{D}_x^\top \mathbf{D}_y^\top] \mathbf{z} = \mathbf{D}_x^\top \mathbf{z}_1 + \mathbf{D}_y^\top \mathbf{z}_2$, with $\mathbf{z}_1, \mathbf{z}_2$ denoting the first and second half of $\mathbf{z}$.) all encode 2D-convolutions with circular boundary conditions.

Exploiting the duality of the signal-processing perspective and the algebraic perspective yields the following reformulations of the denominator, respectively, of the nominator of [\eqref{eq:hqs_tv_1}](#):

$$(\mathbf{H}^\top \mathbf{H} + \rho \mathbf{D}^\top \mathbf{D}) \Leftrightarrow \mathcal{F}^{-1} \left\{ \mathcal{F}\{h\}^* \cdot \mathcal{F}\{h\} + \rho (\mathcal{F}\{d_x\}^* \cdot \mathcal{F}\{d_x\} + \mathcal{F}\{d_y\}^* \cdot \mathcal{F}\{d_y\}) \right\},$$

$$(\mathbf{H}^\top \mathbf{g} + \rho \mathbf{D}^\top \mathbf{z}) \Leftrightarrow \mathcal{F}^{-1} \left\{ \mathcal{F}\{h\}^* \cdot \mathcal{F}\{g\} + \rho (\mathcal{F}\{d_x\}^* \cdot \mathcal{F}\{z_1\} + \mathcal{F}\{d_y\}^* \cdot \mathcal{F}\{z_2\}) \right\},$$

with $(\cdot)^*$ denoting the element-wise complex conjugate.

Combining both terms in the original fraction yields the sought proximal operator:

$$\text{prox}_{\|\cdot\|_2, \rho}(\mathbf{z}) = \mathcal{F}^{-1} \left\{ \frac{\mathcal{F}\{h\}^* \cdot \mathcal{F}\{g\} + \rho (\mathcal{F}\{d_x\}^* \cdot \mathcal{F}\{z_1\} + \mathcal{F}\{d_y\}^* \cdot \mathcal{F}\{z_2\})}{\mathcal{F}\{h\}^* \cdot \mathcal{F}\{h\} + \rho (\mathcal{F}\{d_x\}^* \cdot \mathcal{F}\{d_x\} + \mathcal{F}\{d_y\}^* \cdot \mathcal{F}\{d_y\})} \right\}.$$

As for the inverse filter introduced before, this proximal operator is also unstable with respect to noise and zeros in the involved Fourier transforms. However, the integration into the HQS iterations will mitigate these effects so that the resulting estimate $\hat{\mathbf{s}}$ will not be affected.

Note

All terms of the proximal operator that do not depend on z only have to be computed once and can then be reused.

General regularizer

For a general regularizer, we assume $\mathbf{D}$ to be the identity matrix so that it can be ignored. The proximal operator can then be written as:

$$\text{prox}_{\|\cdot\|_2, \rho}(\mathbf{z}) = \mathcal{F}^{-1} \left\{ \frac{\mathcal{F}\{h\}^* \cdot \mathcal{F}\{g\} + \rho \mathcal{F}\{z\}}{\mathcal{F}\{h\}^* \cdot \mathcal{F}\{h\} + \rho} \right\}.$$

Efficient implementation of z -update for anisotropic TV regularizer

For the z -update, we need to find a solution for the proximal operator

$$\text{prox}_{\|\cdot\|_1, \rho}(\mathbf{Ds}) = \arg \min_{\mathbf{z}} \lambda \|\mathbf{z}\|_1 + \frac{\rho}{2} \|\mathbf{Ds} - \mathbf{z}\|_2^2.$$

To simplify the writing, we substitute $\mathbf{v} = \mathbf{Ds}$:

$$\text{prox}_{\|\cdot\|_1, \rho}(\mathbf{v}) = \arg \min_{\mathbf{z}} \lambda \|\mathbf{z}\|_1 + \frac{\rho}{2} \|\mathbf{v} - \mathbf{z}\|_2^2.$$

We now have to calculate the gradient of that expression with respect to $\mathbf{z}$. To easily follow the derivation, we will work on a single element of the gradient, i.e., we consider the scalar function

$$b(z) = \frac{\rho}{2} (v - z)^2 + \lambda |z|.$$

In order to calculate the gradient $b'(z)$ of $b(z)$, we can work on the two terms of the addition separately. The derivative of the first term $\frac{\rho}{2} (v - z)^2$ is

$$\frac{\partial}{\partial z} \frac{\rho}{2} (v - z)^2 = \rho(-v + z).$$

Unfortunately, the absolute value function $|\cdot|$ (i.e., the 1-norm $\|\cdot\|_1$) is not differentiable and we have to take a detour to solve that problem.

For convex functions there is the concept of subdifferentials and subgradients which can be employed to approximate the gradient of convex functions at positions where they are not differentiable.

Consider a convex function $f: \mathcal{S} \rightarrow \mathbb{R}$ defined on the open interval $\mathcal{S}$. According to Taylor's theorem, the linear approximation of such a function f at any point is strictly smaller than the value of the function at that point itself, i.e.,

$$\forall a \in \mathcal{S} : f(x) \geq f(a) + f'(a)(x - a).$$

This inequality also holds for other values g instead of $f'(a)$, which are smaller than the true gradient $f'(a)$ and since a gradient is never infinite, there have to be infinitely many of suitable g :

$$\forall a \in \mathcal{S} \exists g \in \mathbb{R} : f(x) \geq f(a) + g(x - a).$$

Such values g are called *subgradients* of the function f at position a . The set of all subgradients of f at position a is called the *subdifferential* $\partial_a f$ of f at position a :

$$\partial_a f(x) = \{g \in \mathbb{R} : f(x) \geq f(a) + g(x - a)\}.$$

Subdifferentials have the following useful properties (w.r.t a convex function f as defined above):

- The function f is differentiable at position a if and only if the set of subdifferentials at position a contains only a single slope value, i.e., if $|\partial_a f(x)| = 1$.
- A point a is a global minimum of f if and only if $0 \in \partial_a f(x)$.
- Let $k(x)$ be another convex function like f . With the subdifferentials $\partial_a f(x)$, $\partial_a k(x)$ for some position a , the subdifferential of $f + k$ for position a is then $\partial_a(f + k)(x) = \partial_a f(x) \oplus \partial_a k(x)$ with $\oplus$ denoting the so-called Minkowski sum (i.e., the set of all possible sums between all elements of the two input sets).

We will now derive the subdifferential of $b(z) = \frac{\rho}{2}(v - z)^2 + \lambda|z|$ (neglecting the position a for simplicity) and look for subgradients of 0, which correspond to the sought minimum.

According to the third property introduced before, it is:

$$\partial b(z) = \partial \frac{\rho}{2}(v - z)^2 \oplus \partial \lambda|z|.$$

Of course, the true gradient $f'(a)$ of a function $f(x)$ at position a is also a valid subgradient at position a , i.e.:

$$\partial b(z) = \{\rho(-v + z)\} \oplus \partial \lambda|z|.$$

The subdifferentials for the absolute value function are:

$$\partial \lambda|z| = \begin{cases} -\lambda & \text{if } z < 0 \\ [-\lambda, \lambda] & \text{if } z = 0. \\ \lambda & \text{if } z > 0 \end{cases}$$

For $z \neq 0$, the absolute function is differentiable which is why its subdifferentials for those positions contain only one element (λ or $-\lambda$ respectively). For $z = 0$ there infinitely many possible slope values inside the interval $[-\lambda, \lambda]$ (note: the value 0 is also contained in that interval, indicating that the global minimum must be at position 0).

Combining [\eqref{eq:hqs_z_1}](#) and [\eqref{eq:hqs_z_2}](#) yields:

$$\partial b(z) = \begin{cases} \rho(-v + z) - \lambda & \text{if } z < 0 \\ [-\rho v - \lambda, -\rho v + \lambda] & \text{if } z = 0. \\ \rho(-v + z) + \lambda & \text{if } z > 0 \end{cases}$$

We now equate all cases separately to 0, solve for z and calculate the corresponding interval for v (which is the variable that we know, so we can use it to decide which case applies):

$$\begin{aligned} \textbf{Case 1, } z < 0 : \quad \rho(-v + z) - \lambda &\stackrel{!}{=} 0 \Leftrightarrow z \stackrel{!}{=} v + \frac{\lambda}{\rho} \\ v + \frac{\lambda}{\rho} &\stackrel{!}{<} 0 \Leftrightarrow v < -\frac{\lambda}{\rho}. \end{aligned}$$

For the second case, which is where the sought minimum is located, we want to always return 0 which is why we have to make sure that 0 is contained in the interval of possible slopes, i.e., $0 \in [-\rho v - \lambda, -\rho v + \lambda]$. Hence, it must hold

$$-\rho v - \lambda \leq 0 \Leftrightarrow v \geq -\frac{\lambda}{\rho} \quad \text{and}$$

$$0 \leq -\rho v + \lambda \Leftrightarrow v \leq \frac{\lambda}{\rho},$$

i.e.,

$$\textbf{Case 2, } z < 0 : \quad |v| \leq \frac{\lambda}{\rho}.$$

The third case can be solved analogously to the first case:

$$\textbf{Case 3, } z > 0 : \quad \rho(-v - z) + \lambda \stackrel{!}{=} 0 \Leftrightarrow z \stackrel{!}{=} v - \frac{\lambda}{\rho}$$

$$v - \frac{\lambda}{\rho} \stackrel{!}{>} 0 \Leftrightarrow v > \frac{\lambda}{\rho}.$$

□

The sought proximal operator is given by

$$\text{prox}_{\|\cdot\|_{1,\rho}}(\mathbf{v})_i = \mathcal{S}_{\lambda/\rho}(v_i) = \begin{cases} v_i + \frac{\lambda}{\rho} & \text{if } v_i < -\frac{\lambda}{\rho} \\ 0 & \text{if } |v_i| \leq \frac{\lambda}{\rho} \\ v_i - \frac{\lambda}{\rho} & \text{if } v_i > \frac{\lambda}{\rho} \end{cases},$$

with v_i referring to the i -th element of the corresponding vector $\mathbf{v}$ (etc.).

The operator $\mathcal{S}$ is also called the element-wise *soft thresholding operator*.

Visualization of soft thresholding operator

```
def y(z,v, rho, lam):
    return lam*np.abs(z) + rho/2.0 * np.linalg.norm(v-z)**2
```

```
def s(v,kap):
    if v > kap:
        return v-kap
    if v < (-1*kap):
        return v+kap
    else:
        return 0
```

```
def soft_thres_visu(v):
    rho = 0.5
    lam = 0.4

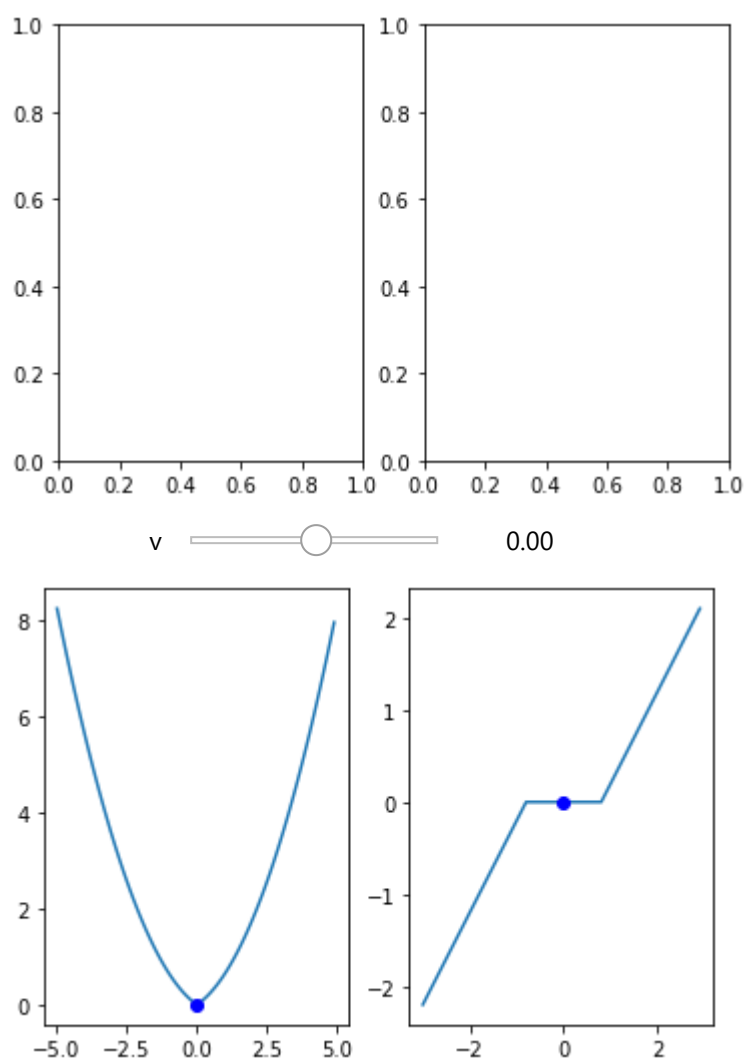
    zs = np.arange(-5.0,5.0,0.1)
    ys = [y(z,v,rho,lam) for z in zs]

    vs = np.arange(-3.0,3.0,0.1)
    ss = [s(v,lam/rho) for v in vs]

    #plt.subplots(1,2)
    plt.clf()
    plt.subplot(1,2,1)
    plt.plot(zs, ys)
    plt.plot(cur_s:=s(v, lam/rho), y(cur_s, v, rho, lam), 'bo' )

    plt.subplot(1,2,2)
    plt.plot(vs, ss)
    plt.plot(v,s(v,lam/rho),'bo')
```

```
plt.subplots(1,2)
interact(lambda v: soft_thres_visu(v), v=widgets.FloatSlider(min=-3.0,max=3.0,
step=0.2, value=0))
```



```
<function __main__.<lambda>(v)>
```

Efficient implementation of z -update for isotropic TV regularizer

For the isotropic case, the l_2 -norm of the finite differences approximation of the horizontal and vertical image gradients is used as the regularizer instead of the l_1 -norm, i.e.,

$$\lambda \| \mathbf{z} \|_{2,1} = \lambda \sum_{i=1}^N \left\| \begin{bmatrix} (\mathbf{D}_x \mathbf{s})_i \\ (\mathbf{D}_y \mathbf{s})_i \end{bmatrix} \right\|_2 = \lambda \sum_{i=1}^N \sqrt{(\mathbf{D}_x \mathbf{s})_i^2 + (\mathbf{D}_y \mathbf{s})_i^2}.$$

This expression is also called the *group lasso*.

Accordingly, the whole deconvolution problem for the isotropic TV regularizer is

$$\arg \min_{\mathbf{s}} \underbrace{\frac{1}{2} \|\mathbf{H}\mathbf{s} - \mathbf{g}\|_2^2}_{=:f(\mathbf{s})} + \lambda \underbrace{\sum_{i=1}^N \left\| \begin{bmatrix} z_i \\ z_{i+N} \end{bmatrix} \right\|_2}_{=:r(\mathbf{z})}$$

subject to $\mathbf{D}\mathbf{s} - \mathbf{z} = \mathbf{0}$.

Again, the vector $\mathbf{z} \in \mathbb{R}^{2 \times N}$ so that its first, respectively, second half can hold the image gradients in x -direction, respectively, in y -direction, i.e., $\mathbf{z} = [\mathbf{D}_x \mathbf{s} \ \mathbf{D}_y \mathbf{s}]^\top$.

As only the regularization term $r(\mathbf{z})$ has changed, we only have to derive a corresponding $\mathbf{z}$ -update rule, i.e., we have to find a solution for the proximal operator

$$\mathbf{z} \leftarrow \mathbf{prox}_{\|\cdot\|_{2,1}, \rho}(\mathbf{v}) = \arg \min_{\mathbf{z}} \lambda \sum_{i=1}^N \left\| \begin{bmatrix} z_i \\ z_{i+N} \end{bmatrix} \right\|_2 + \frac{\rho}{2} \|\mathbf{v} - \mathbf{z}\|_2^2, \quad \mathbf{v} = \mathbf{D}\mathbf{s}.$$

Therefore, we again extract the term we have to minimize:

$$b(\mathbf{z}) := \lambda \|\mathbf{z}\|_2 + \frac{\rho}{2} \|\mathbf{v} - \mathbf{z}\|_2^2.$$

We follow the approach of calculating the gradient of $b(\mathbf{z})$, equating it to $\mathbf{0}$ and solving for $\mathbf{z}$.

The gradient of the second term with respect to $\mathbf{z}$ can be calculated straight forward:

$$\nabla_{\mathbf{z}} \frac{\rho}{2} \|\mathbf{v} - \mathbf{z}\|_2^2 = \rho(-\mathbf{v} + \mathbf{z}).$$

The gradient of the Euclidean norm $\|\mathbf{x}\|_2$ is

$$\nabla_{\mathbf{x}} \|\mathbf{x}\|_2 = \frac{\mathbf{x}}{\|\mathbf{x}\|_2}, \quad \text{for } \mathbf{x} \neq \mathbf{0}.$$

Hence, the gradient of the first term of \eqref{eq:hqs:iso:1} is

$$\nabla_{\mathbf{z}} \lambda \|\mathbf{z}\|_2 = \lambda \frac{\mathbf{z}}{\|\mathbf{z}\|_2} \quad \text{for } \mathbf{z} \neq \mathbf{0}.$$

So for $\mathbf{z} \neq \mathbf{0}$, we will find a $\hat{\mathbf{z}}$ with $\nabla_{\mathbf{z}} \{b(\mathbf{z})\}(\hat{\mathbf{z}}) = \mathbf{0}$:

$$\begin{aligned} \lambda \frac{\mathbf{z}}{\|\mathbf{z}\|_2} + \rho(-\mathbf{v} + \mathbf{z}) &\stackrel{!}{=} \mathbf{0} \\ \mathbf{z} \left(\frac{\lambda}{\|\mathbf{z}\|_2} + \rho \right) &= \rho \mathbf{v} \\ \mathbf{z} \left(\frac{\lambda}{\|\mathbf{z}\|_2 \rho} + 1 \right) &= \mathbf{v}. \end{aligned}$$

We now apply the l_2 -norm to both sides of the equation:

$$\left| \frac{\lambda}{\|\mathbf{z}\|_2 \rho} + 1 \right| \cdot \|\mathbf{z}\|_2 = \|\mathbf{v}\|_2.$$

The last step introduced an absolute value term for which the two possible cases, .i.e, > 0 , < 0 would have to be considered in principle. However, since the whole equation represents a relation between two vector norms like $\|\mathbf{a}\|_2 \cdot \alpha = \|\mathbf{b}\|_2$ where $\alpha \geq 0$ always because $\|\cdot\|_2 \geq 0$ always, we can conclude that

$$\frac{\lambda}{\|\mathbf{z}\|_2 \rho} + 1 > 0,$$

leading to

$$\begin{aligned} \frac{\lambda}{\|\mathbf{z}\|_2 \rho} + 1 &> 0 \\ \frac{\lambda}{\|\mathbf{z}\|_2} &> -\rho \\ \frac{1}{\|\mathbf{z}\|_2} &> -\frac{\rho}{\lambda} \\ -\frac{\lambda}{\rho} &< \|\mathbf{z}\|_2, \end{aligned}$$

which we need again later.

We continue to solve \eqref{eq:hqs:iso:3} for $\|\mathbf{z}\|_2$:

$$\begin{aligned} \left(\frac{\lambda}{\|\mathbf{z}\|_2} + \rho \right) \cdot \|\mathbf{z}\|_2 &= \rho \|\mathbf{v}\|_2 \\ \lambda + \rho \|\mathbf{z}\|_2 &= \rho \|\mathbf{v}\|_2 \\ \|\mathbf{z}\|_2 &= \frac{\rho \|\mathbf{v}\|_2 - \lambda}{\rho}, \end{aligned}$$

and now insert this expression for $\|\mathbf{z}\|_2$ into \eqref{eq:hqs:iso:2} and solve for $\mathbf{z}$:

$$\begin{aligned} \mathbf{z} \left(\lambda \cdot \frac{\rho}{\rho \|\mathbf{v}\|_2 - \lambda} + \rho \right) &= \rho \mathbf{v} \\ \mathbf{z} \frac{\lambda + \rho \|\mathbf{v}\|_2 - \lambda}{\rho \|\mathbf{v}\|_2 - \lambda} &= \mathbf{v} \\ \mathbf{z} &= \mathbf{v} \cdot \frac{\rho \|\mathbf{v}\|_2 - \lambda}{\rho \|\mathbf{v}\|_2} \\ \mathbf{z} &= \mathbf{v} \cdot \left(1 - \frac{\lambda}{\rho \|\mathbf{v}\|_2} \right). \end{aligned}$$

In order to make sure that $\mathbf{z} \neq \mathbf{0}$, we have to check for which conditions $\left(1 - \frac{\lambda}{\rho \|\mathbf{v}\|_2} \right)$ is greater or less than 0. Let's assume that is less than zero and perform some manipulations:

$$\begin{aligned}
1 - \frac{\lambda}{\rho \|\mathbf{v}\|_2} &< 0 \\
-\frac{\lambda}{\rho \|\mathbf{v}\|_2} &< -1 \\
\frac{\lambda}{\rho \|\mathbf{v}\|_2} &> 1 \\
\frac{\lambda}{\rho} &> \|\mathbf{v}\|_2.
\end{aligned}$$

From \eqref{eq:hqs:ios:11} we know that $\|\mathbf{v}\|_2$ can be expressed as

$$\|\mathbf{v}\|_2 = \|\mathbf{z}\|_2 \left(\frac{\lambda}{\|\mathbf{z}\|_2 \rho} + 1 \right) = \frac{\lambda}{\rho} + \|\mathbf{z}\|_2,$$

we can continue our previous thoughts

$$\begin{aligned}
\frac{\lambda}{\rho} &> \|\mathbf{v}\|_2 \\
\frac{\lambda}{\rho} &> \frac{\lambda}{\rho} + \|\mathbf{z}\|_2 \\
0 &> \|\mathbf{z}\|_2,
\end{aligned}$$

what is impossible since $\|\cdot\|_2 > 0$ always. Hence, we can focus on the other case:

$$\begin{aligned}
1 - \frac{\lambda}{\rho \|\mathbf{v}\|_2} &> 0 \\
\|\mathbf{v}\|_2 &> \frac{\lambda}{\rho}.
\end{aligned}$$

For $\mathbf{z} = \mathbf{0}$, there is no well-defined gradient for $\|\mathbf{z}\|_2$, so we make use of the concept of subdifferentials again and derive the subdifferential of $b(\mathbf{z})$ with respect to position $\mathbf{z} = \mathbf{0}$:

$$\begin{aligned}
b(\mathbf{z}) &= \lambda \|\mathbf{z}\|_2 + \frac{\rho}{2} \|\mathbf{v} - \mathbf{z}\|_2^2 \\
\frac{b(\mathbf{z})}{\rho} &= \frac{\lambda}{\rho} \|\mathbf{z}\|_2 + \frac{1}{2} \|\mathbf{v} - \mathbf{z}\|_2^2 \\
\partial_0 \frac{b(\mathbf{z})}{\rho} &= \partial_0 \frac{\lambda}{\rho} \|\mathbf{z}\|_2 \oplus \partial_0 \frac{1}{2} \|\mathbf{v} - \mathbf{z}\|_2^2.
\end{aligned}$$

As before, we can replace the second subdifferential on the right side of the equation with the true gradient:

$$\partial_0 \frac{b(\mathbf{z})}{\rho} = \partial_0 \frac{\lambda}{\rho} \|\mathbf{z}\|_2 \oplus \{-\mathbf{v}\}.$$

For the first term, i.e., $\partial_0 \frac{\lambda}{\rho} \|\mathbf{z}\|_2$, we employ the definition of subdifferentials:

$$\begin{aligned}
\partial_0 \frac{\lambda}{\rho} \|\mathbf{z}\|_2 &= \left\{ \mathbf{g} \in \mathbb{R}^{2N} : \frac{\lambda}{\rho} \|\mathbf{z}\|_2 \geq \frac{\lambda}{\rho} \|\mathbf{0}\|_2 + \langle \mathbf{g}, \mathbf{z} - \mathbf{0} \rangle \right\} \\
\partial_0 \frac{\lambda}{\rho} \|\mathbf{z}\|_2 &= \left\{ \mathbf{g} \in \mathbb{R}^{2N} : \frac{\lambda}{\rho} \|\mathbf{z}\|_2 \geq \langle \mathbf{g}, \mathbf{z} \rangle \right\}.
\end{aligned}$$

By means of the Cauchy-Schwarz inequality, i.e., $\langle \mathbf{g}, \mathbf{z} \rangle \leq \|\mathbf{g}\| \cdot \|\mathbf{z}\|$, we can further simplify the derivation:

$$\begin{aligned}
\partial_0 \frac{\lambda}{\rho} \|\mathbf{z}\|_2 &= \left\{ \mathbf{g} \in \mathbb{R}^{2N} : \frac{\lambda}{\rho} \|\mathbf{z}\|_2 \geq \langle \mathbf{g}, \mathbf{z} \rangle \right\} \\
&= \left\{ \mathbf{g} \in \mathbb{R}^{2N} : \frac{\lambda}{\rho} \|\mathbf{z}\|_2 \geq \|\mathbf{g}\|_2 \cdot \|\mathbf{z}\|_2 \right\} \\
&= \left\{ \mathbf{g} \in \mathbb{R}^{2N} : \frac{\lambda}{\rho} \geq \|\mathbf{g}\|_2 \right\}.
\end{aligned}$$

The resulting combined expression for the subdifferential of $\frac{b(\mathbf{z})}{\rho}$ is:

$$\partial_0 \frac{b(\mathbf{z})}{\rho} = \left\{ \mathbf{g} \in \mathbb{R}^{2N} : \frac{\lambda}{\rho} \geq \|\mathbf{g}\|_2 \right\} \oplus \{-\mathbf{v}\}.$$

For $\mathbf{z} = \mathbf{0}$ we want to return $\mathbf{0}$ if possible since that would correspond to the sought minimum.

A subgradient $\mathbf{0}$ is contained in $\partial_{\mathbf{0}} \frac{b(\mathbf{z})}{\rho}$ if the first subdifferential $\left\{ \mathbf{g} \in \mathbb{R}^{2N} : \frac{\lambda}{\rho} \geq \|\mathbf{g}\|_2 \right\}$

contains a vector $\mathbf{g}$ with $\mathbf{g} = \mathbf{v}$ because then the Minkowski sum

$\left\{ \mathbf{g} \in \mathbb{R}^{2N} : \frac{\lambda}{\rho} \geq \|\mathbf{g}\|_2 \right\} \oplus \{-\mathbf{v}\}$ would contain $\mathbf{0}$. This is only possible for any vectors $\mathbf{v}$ with $\|\mathbf{v}\|_2 \leq \frac{\lambda}{\rho}$.

With these results, the final $\mathbf{z}$ -update rule is given by:

$$\begin{bmatrix} z_i \\ z_{i+N} \end{bmatrix} \leftarrow \mathcal{S}_{\lambda/\rho} \left(\mathbf{v}' := \begin{bmatrix} v_i \\ v_{i+N} \end{bmatrix} \right) = \begin{cases} \mathbf{v}' \cdot \left(1 - \frac{\lambda}{\rho \|\mathbf{v}'\|_2} \right) & \text{if } \|\mathbf{v}'\|_2 > \frac{\lambda}{\rho} \\ \mathbf{0} & \text{if } \|\mathbf{v}'\|_2 \leq \frac{\lambda}{\rho} \end{cases}.$$

This expression (i.e., $\mathcal{S}_{\lambda/\rho}$) is also known as the vector soft-thresholding operator.

Efficient update of $\mathbf{z}$ in general case

In the general case, i.e., with $\mathbf{D} = \mathbf{I}$, we want to solve

$$\begin{aligned} \mathbf{z} &\leftarrow \arg \min_{\mathbf{z}} r(\mathbf{z}) + \frac{\rho}{2} \|\mathbf{s} - \mathbf{z}\|_2^2, \\ &\arg \min_{\mathbf{z}} \lambda \Psi(\mathbf{z}) + \frac{\rho}{2} \|\mathbf{s} - \mathbf{z}\|_2^2, \\ &\arg \min_{\mathbf{z}} \Psi(\mathbf{z}) + \frac{\rho}{2\lambda} \|\mathbf{s} - \mathbf{z}\|_2^2. \end{aligned}$$

For $\frac{\lambda}{\rho} = \sigma^2$, the last equation represents a denoising problem for zero-mean Gaussian noise with variance σ^2 as we have seen earlier:

$$\arg \min_{\mathbf{s}} \Gamma(\mathbf{s}) + \frac{1}{2\sigma^2} \|\mathbf{g} - \mathbf{s}\|_2^2.$$

This means that we can use any Gaussian denoiser $\mathcal{D} : \mathbb{R}^N \rightarrow \mathbb{R}^N$ as our proximal operator for the z -update, e.g., the denoising CNN from the last chapter.

→ Any Gaussian denoiser suitable for a noise variance of σ^2 can be used as a general image prior:

$$\mathbf{z} \leftarrow \text{prox}_{\mathcal{D}, \rho}(\mathbf{s}) = \mathcal{D} \left(\mathbf{s}, \sigma^2 = \frac{\lambda}{\rho} \right).$$

HQS for general linear inverse problems

HQS can also be applied to linear inverse problems, i.e., $\mathbf{g} = \mathbf{H}\mathbf{s} + \mathbf{n}$.

Depending on the prior, we can use the $\mathbf{z}$ -updates already derived for the deconvolution problem (i.e., total variation or a general Gaussian denoiser).

However, we have to look again at the $\mathbf{s}$ -update, as the matrix-vector multiplication $\mathbf{H}\mathbf{s}$ does not (necessarily) encode a convolution anymore.

Derivation of $\mathbf{s}$ -update for HQS for general linear inverse problems

For the $\mathbf{s}$ -update, we use the following solution:

$$\begin{aligned} \mathbf{s} &\leftarrow \text{prox}_{\|\cdot\|_2, \rho}(\mathbf{v}) = \arg \min_{\mathbf{s}} \frac{1}{2} \|\mathbf{H}\mathbf{s} - \mathbf{g}\|_2^2 + \frac{\rho}{2} \|\mathbf{D}\mathbf{s} - \mathbf{z}\|_2^2 \\ &= \left(\underbrace{\mathbf{H}^\top \mathbf{H} + \rho \mathbf{D}^\top \mathbf{D}}_{\tilde{\mathbf{H}}} \right)^{-1} \left(\underbrace{\mathbf{H}^\top \mathbf{g} + \rho \mathbf{D}^\top \mathbf{z}}_{\tilde{\mathbf{g}}} \right), \end{aligned}$$

whose derivation is equivalent to the deconvolution problem.

Unfortunately, for the case of a general matrix $\mathbf{H}$, there is no elegant closed-form solution as for the case of deconvolution.

→ Use an iterative solver to solve $\tilde{\mathbf{H}}\mathbf{s} = \tilde{\mathbf{g}}$ for $\mathbf{s}$. Since $\tilde{\mathbf{H}}$ is symmetric and positive semi-definite, the so-called *conjugate gradient* method is a good choice.

Problems of HQS

The HQS method works well for inverse problems where the number of available measurements M is in the range of the number of unknowns N . For the case of deconvolution, it is $M = N$ and hence HQS represents a suitable choice.

However, for severely under-determined problems, the penalty $\frac{\rho}{2}\|\mathbf{Ds} - \mathbf{z}\|_2^2$ linking data fidelity and the prior terms can be too weak so that the algorithm does not converge well.

We will now get to know a method that mitigates that problem.

Alternating direction method of multipliers (ADMM)

Similar to the HQS approach, the so-called *alternating direction method of multipliers (ADMM)* splits the objective function

$$\arg \min_{\mathbf{s}} \underbrace{\frac{1}{2}\|\mathbf{H}\mathbf{s} - \mathbf{g}\|_2^2}_{=:f(\mathbf{s})} + \underbrace{\lambda\Psi(\mathbf{z})}_{=:r(\mathbf{z})}$$

subject to $\mathbf{Ds} - \mathbf{z} = \mathbf{0}$.

into multiple terms and optimizes it in an alternating fashion.

But instead of transforming the constraint $\mathbf{Ds} - \mathbf{z} = \mathbf{0}$ into a penalty term, it employs the so-called *augmented Lagrangian* of the objective function:

$$L_{\rho}^{(\text{ADMM})}(\mathbf{s}, \mathbf{z}, \mathbf{y}) = f(\mathbf{s}) + r(\mathbf{z}) + \mathbf{y}^T (\mathbf{Ds} - \mathbf{z}) + \frac{\rho}{2}\|\mathbf{Ds} - \mathbf{z}\|_2^2,$$

with the Lagrange multiplier $\mathbf{y}$.

The so-called *scaled form* of the augmented Lagrangian is given by

$$L_{\rho}^{(\text{ADMM})}(\mathbf{s}, \mathbf{z}, \mathbf{u}) = f(\mathbf{s}) + r(\mathbf{z}) + \frac{\rho}{2}\|\mathbf{Ds} - \mathbf{z} + \mathbf{u}\|_2^2 - \frac{\rho}{2}\|\mathbf{u}\|_2^2,$$

with the scaled Lagrangian multiplier $\mathbf{u} = \frac{1}{\rho}\mathbf{y}$.

Note

The augmented Lagrangian and its scaled form are identical, i.e., just mathematical reformulations that come in handy for further steps.

The update rules for the variables $\mathbf{s}, \mathbf{z}, \mathbf{u}$ are given by

$$\begin{aligned} \mathbf{s} &\leftarrow \text{prox}_{\|\cdot\|_2, \rho}(\mathbf{v}) = \arg \min_{\mathbf{s}} L_{\rho}^{(\text{ADMM})}(\mathbf{s}, \mathbf{z}, \mathbf{u}) = \arg \min_{\mathbf{s}} \frac{1}{2}\|\mathbf{H}\mathbf{s} - \mathbf{g}\|_2^2 + \frac{\rho}{2}\|\mathbf{Ds} - \mathbf{z} + \mathbf{u}\|_2^2, \\ \mathbf{z} &\leftarrow \text{prox}_{\Psi, \rho}(\mathbf{v}) = \arg \min_{\mathbf{z}} L_{\rho}^{(\text{ADMM})}(\mathbf{s}, \mathbf{z}, \mathbf{u}) = \arg \min_{\mathbf{z}} \lambda\Psi(\mathbf{z}) + \frac{\rho}{2}\|\mathbf{Ds} - \mathbf{z} + \mathbf{u}\|_2^2, \\ \mathbf{u} &\leftarrow \mathbf{u} + \mathbf{Ds} - \mathbf{z} \end{aligned}$$

For the $\mathbf{s}$ - and $\mathbf{z}$ -updates we can again use proximal operators. The $\mathbf{u}$ -update is a straightforward sum of vectors (for its derivation see the article [Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers](#) by Stephen Boyd et al.).

The **s**-update:

The **s**-update within ADMM is quite similar to that of HQS, we just have to adequately account for **u**, i.e.,

$$\text{prox}_{\|\cdot\|_2, \rho}(\mathbf{z} - \mathbf{u}) = \left(\underbrace{\mathbf{H}^\top \mathbf{H} + \rho \mathbf{D}^\top \mathbf{D}}_{\tilde{\mathbf{H}}} \right)^{-1} \left(\underbrace{\mathbf{H}^\top \mathbf{g} + \rho \mathbf{D}^\top (\mathbf{z} - \mathbf{u})}_{\tilde{\mathbf{g}}} \right).$$

The **z**-update:

For the **z**-update, we can rely on the same proximal operators as HQS.

In the case of the TV regularizer, the proximal operator is the element-wise soft thresholding operator:

$$\text{prox}_{\|\cdot\|_1, \rho}(\mathbf{v}) = \arg \min_{\mathbf{z}} \lambda \|\mathbf{z}\|_1 + \frac{\rho}{2} \|\mathbf{v} - \mathbf{z}\|_2^2 = \mathcal{S}_{\lambda/\rho}(\mathbf{v}).$$

Since for HQS it is $\mathbf{v} = \mathbf{D}\mathbf{s}$, we again have to account for **u** as $\mathbf{v} = \mathbf{D}\mathbf{s} + \mathbf{u}$.

In the general case, i.e., for $\mathbf{D} = \mathbf{I}$, we can use any Gaussian denoiser $\mathcal{D} : \mathbb{R}^N \rightarrow \mathbb{R}^N$ as a regularizer leading to the following **z**-update:

$$\text{prox}_{\mathcal{D}, \rho}(\mathbf{v}) = \mathcal{D}\left(\mathbf{v}, \sigma^2 = \frac{\lambda}{\rho}\right),$$

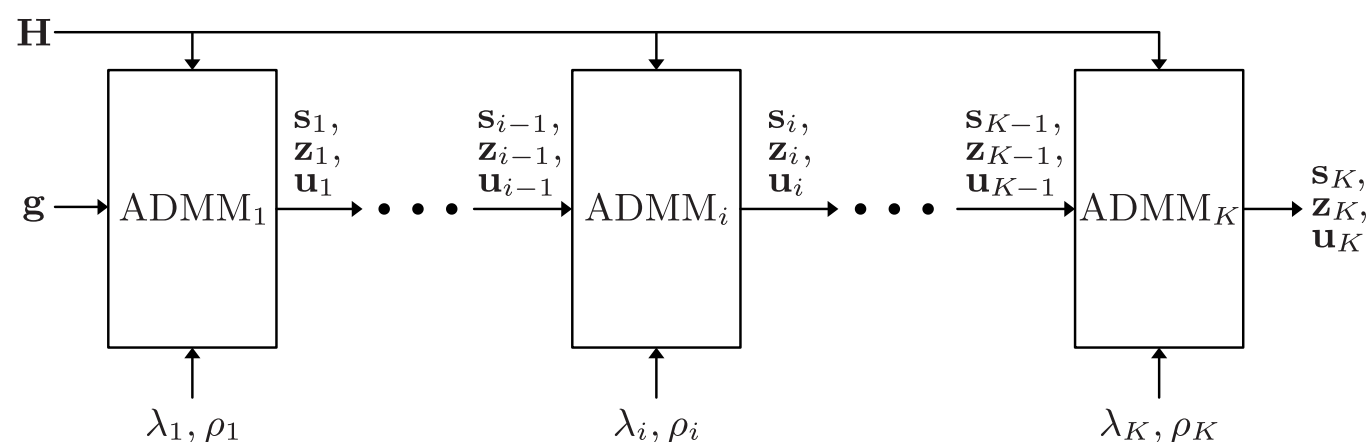
with $\mathbf{v} = \mathbf{s} + \mathbf{u}$.

Unrolled optimization

For traditional HQS and ADMM implementations, the values of the hyperparameters λ and ρ are chosen once and remain fix while performing the optimization.

Although this works well in many cases, it can be beneficial to set the number i of iterations to a fixed value and replicate the parameter update routines i times, each with individual hyperparameters λ_i, ρ_i .

The resulting structure can be interpreted and dealt with like a neural network (especially if some deep denoising prior is included). Then, the hyperparameters λ_i, ρ_i and the parameters of a potentially included neural network can be learned (i.e., optimized) in an end-to-end fashion in a joint training procedure.



Advantages of unrolled optimization:

- Whole method becomes end-to-end differentiable.
- Hyperparameters λ_i, ρ_i can be learned and optimized allowing to realize a hyperparameter-schedule (e.g., increasing the ρ_i over time).
- The denoising neural network can adapt to the actual matrix $\mathbf{A}$ or the convolution kernel (even with respect to every individual iteration).
- Skip connections from earlier iterations to later iterations can be realized.

Deep SNR-estimation for Wiener filtering

As elaborated before, one main drawback of the Wiener filter is the necessity for the user to correctly estimate the signal-to-noise ratio

$$SNR(\mathbf{f}) = \frac{S_{ss}(\mathbf{f})}{S_{nn}(\mathbf{f})},$$

with $S_{ss}(\mathbf{f})$, $S_{nn}(\mathbf{f})$ denoting the power spectral density of the signal component, respectively, of the noise component.

In the unusual case when one has access to s and n , it is possible to calculate (not unbiased) estimates of $S_{ss}(\mathbf{f})$, $S_{nn}(\mathbf{f})$ via

$$\begin{aligned}\hat{S}_{ss}(\mathbf{f}) &= |\mathcal{F}\{s(\mathbf{x})\}|^2, \\ \hat{S}_{nn}(\mathbf{f}) &= |\mathcal{F}\{n(\mathbf{x})\}|^2.\end{aligned}$$

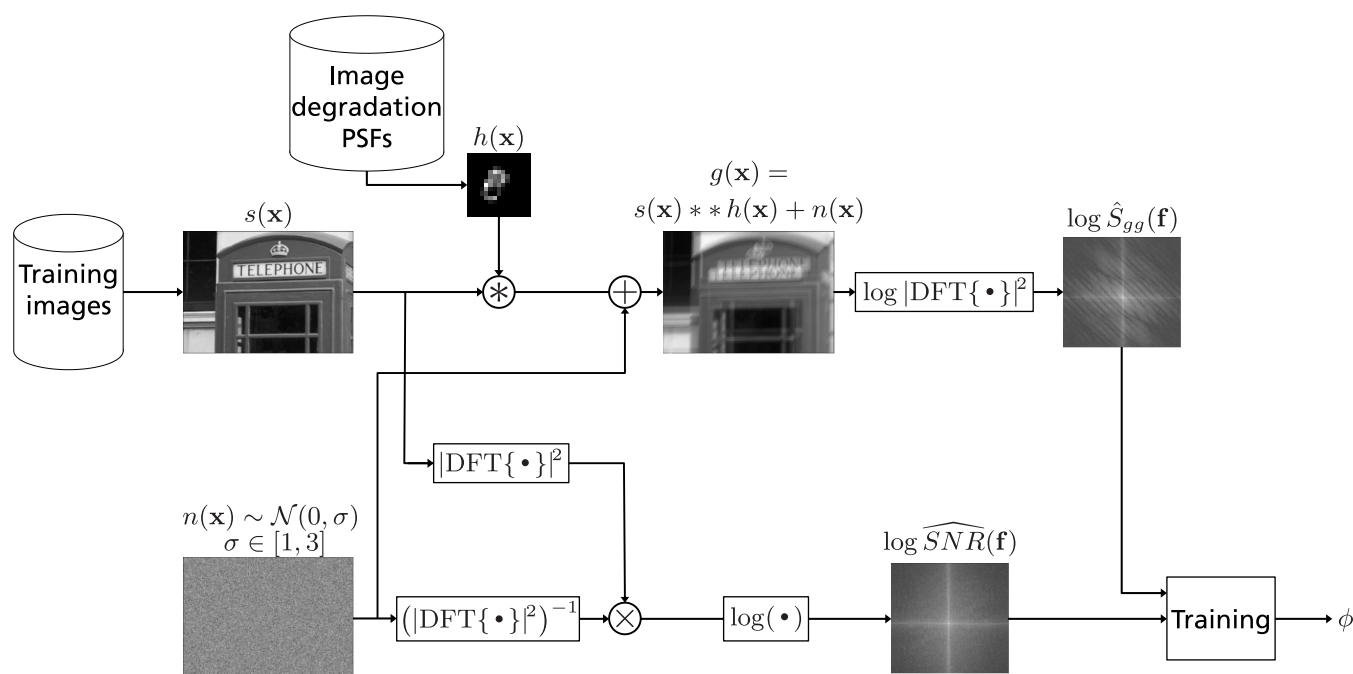
Fortunately we could show that a neural network ϕ can be trained to precisely estimate the signal-to-noise ratio based on the observed image g only.

To make the task easier for the neural network, we trained it to consume estimates of the power spectral density of g to estimate the signal-to-noise ratio. By this means, both input and output have spatial frequencies as variables.

Directly training ϕ with pairs of $(\hat{S}_{gg}, \widehat{SNR})$ would lead to slow or even no convergence of the training due to the high dynamic range of the squared magnitude spectra of the Fourier transforms.

Hence, we train ϕ with pairs $(\log \hat{S}_{gg}, \log \widehat{SNR})$, i.e., so that $\phi(\log \hat{S}_{gg}(\mathbf{f})) = \log \widehat{SNR}(\mathbf{f})$.

The following figure visualizes the training procedure:



To apply the method for image reconstruction the following steps are performed:

- Calculate $\hat{S}_{gg}(\mathbf{f}) = |\mathcal{F}\{g(\mathbf{x})\}|^2$ for the observed image g ,
- use ϕ to estimate the signal-to-noise ratio via $\widehat{SNR}(\mathbf{f}) \leftarrow \phi(\hat{S}_{gg}(\mathbf{f}))$,
- estimate the undistorted image $\hat{s}$ by applying the Wiener filter to g and by using the estimated $\widehat{SNR}$.

Example Wiener filter reconstruction results with deep SNR-estimation

```
interact(lambda i: showFig('figures/6/dw_ex_', i, '.png', 800, 50),
i=widgets.IntSlider(min=(min_i:=1), max=(max_i:=8), step=1, value=(6 if book else min_i)))
```

